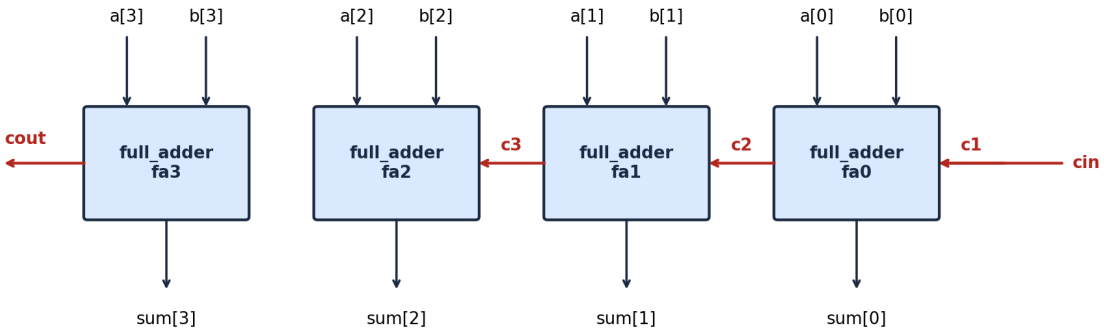
Digital Arithmetic Logic Circuit Library
Using Verilog HDL

Project Documentation - VLSI Design & Semiconductor Engineering Internship, Task 1

RTL Block Diagram - 4-bit Ripple Carry Adder (ripple_carry_adder_4bit)



Carry ripples from fa0 (right, LSB) to fa3 (left, MSB). Critical path = 4 x (full-adder carry delay).

Name	T R JYOTHIRMAY DIKSHITH
Intern ID	SG2884
Date	18 October 2026
Tools	Ubuntu/WSL, Icarus Verilog, GTKWave, VS Code, Git/GitHub
Language	Verilog HDL (IEEE 1364-2001 style)

1. Introduction and Objectives

In this task I built a small library of basic digital circuits in Verilog: logic gates, half and full adders, 2:1 and 4:1 multiplexers, a 4-bit ripple carry adder, and an N-bit adder whose width can be changed with a parameter. Every module has its own testbench that checks the outputs automatically, and I opened the waveforms in GTKWave to confirm they match the truth tables. I plan to reuse these blocks later when I build an ALU and a processor.

Objectives

• Understand the VLSI design flow and where front-end RTL design fits in.

• Use Boolean algebra and K-maps to simplify logic.

• Write clean Verilog that can be synthesised, with consistent naming.

• Check each design with a self-checking testbench, Icarus Verilog and GTKWave.

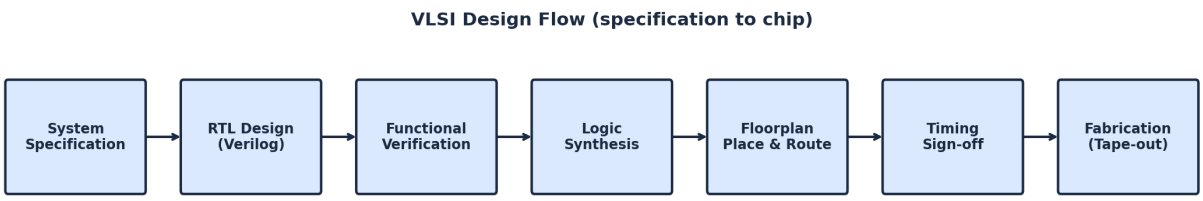
• Keep the project tidy with a clear folder structure, a Makefile and Git.

2. Semiconductor and VLSI Fundamentals (Part A)

2.1 Semiconductor industry and Moore's Law

Chips sit inside almost everything we use: phones, laptops, cars, medical devices and AI hardware. In 1965 Gordon Moore noticed that the number of components on a chip was doubling about every year, and in 1975 he changed that to about every two years. This is known as Moore's Law. It held for a long time because transistors kept getting smaller. These days the gains also come from better chip architecture, packaging and special-purpose accelerators.

2.2 VLSI design flow



Front-end design is the part this internship focuses on: writing the specification, planning the architecture, coding the RTL, simulating it, and synthesising it into gates. **Back-end (physical) design** comes after that. It covers floorplanning, placement, clock tree, routing and the final timing checks, and ends with tape-out to the foundry. An **SoC** (System-on-Chip) puts processor cores, memory, buses and peripherals together on a single die.

2.3 ASIC vs FPGA

Aspect	ASIC	FPGA
Definition	Custom chip fixed at manufacture for one application	Reconfigurable chip (LUTs, flip-flops, block RAM, DSP) programmed after manufacture
Performance / power	Highest speed, lowest power per function	Slower and more power per function (routing overhead)
NRE (one-time) cost	Very high (masks, tools, verification) - often millions of USD at advanced nodes	Low - buy a board, use vendor tools
Unit cost	Very low at high volume	Higher per unit
Time to market	Long (many months to years)	Short (days to weeks)
Flexibility	None after tape-out	Field re-programmable
Typical volume	Millions of units	Prototypes, low/medium volume, evolving standards
Industry examples	Smartphone SoCs (Apple A-series, Qualcomm Snapdragon), GPUs (NVIDIA), Google TPU, network switch chips (Broadcom)	AMD (Xilinx) Zynq/Versal, Intel (Altera) Agilex, Lattice devices; telecom/radar/test equipment; prototyping platforms for ASIC verification
Design tools	Synopsys, Cadence, Siemens EDA flows	AMD Vivado, Intel Quartus (plus open-source flows)

Both start from the same RTL (Verilog/VHDL); an FPGA is frequently used to prototype a design before committing it to an ASIC.

3. Digital Electronics Fundamentals (Part B)

3.1 Number systems and binary arithmetic

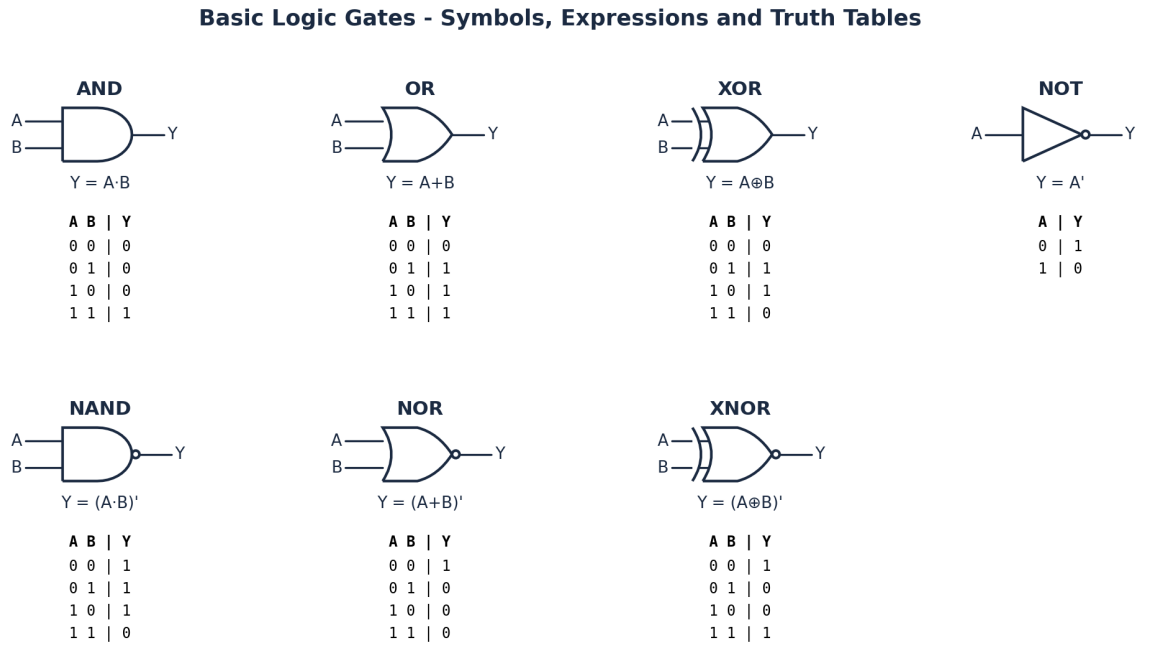
Decimal	Binary (4-bit)	Octal	Hex
0	0000	0	0
1	0001	1	1
2	0010	2	2
5	0101	5	5
9	1001	11	9
10	1010	12	A
15	1111	17	F

Binary addition works like decimal addition, but with only two digits: 0+0=0, 0+1=1, 1+0=1 and 1+1=10 (write 0, carry 1). For example, 0101 (5) + 0011 (3) = 1000 (8). Negative numbers are stored in two's complement: flip every bit and add 1, so -3 in 4 bits is 1101.

3.2 Boolean algebra - key laws

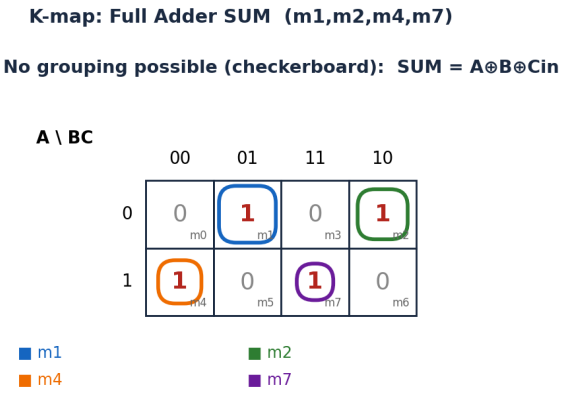
Law	Form
Identity	A + 0 = A ; A . 1 = A
Null	A + 1 = 1 ; A . 0 = 0
Idempotent	A + A = A ; A . A = A
Complement	A + A' = 1 ; A . A' = 0
Commutative / Associative	A + B = B + A ; (A + B) + C = A + (B + C)
Distributive	A . (B + C) = AB + AC ; A + BC = (A + B)(A + C)
Absorption	A + AB = A ; A(A + B) = A
De Morgan	(A . B)' = A' + B' ; (A + B)' = A' . B'
XOR	A XOR B = A'B + AB'

3.3 Logic gates and truth tables

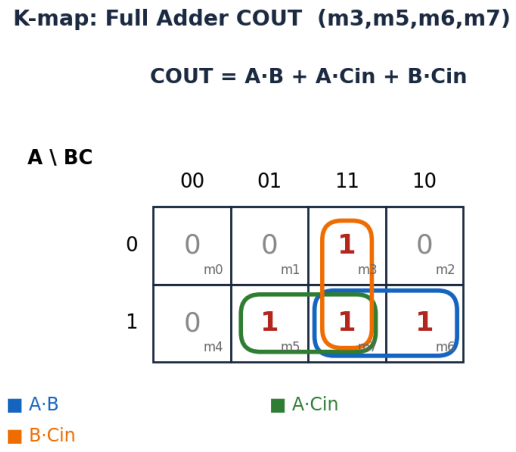


3.4 Karnaugh-map simplification (Exercise 5)

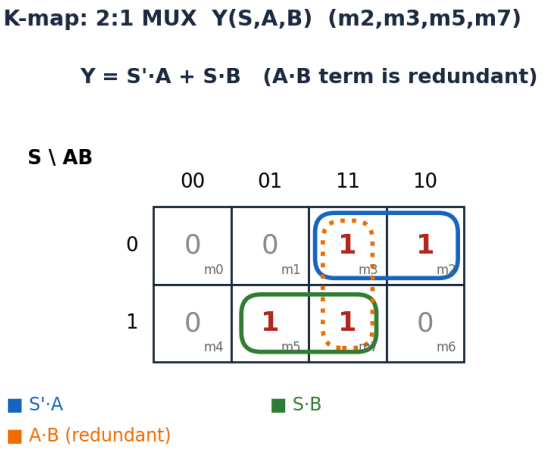
To simplify with a K-map, I write the truth table first and then place the 1s on a grid where neighbouring cells differ in only one variable (Gray code order). I circle the biggest rectangles of 1s whose size is a power of 2, and each circle gives one product term. I solved four circuits this way and then checked every simplified expression against its truth table to make sure it was correct.



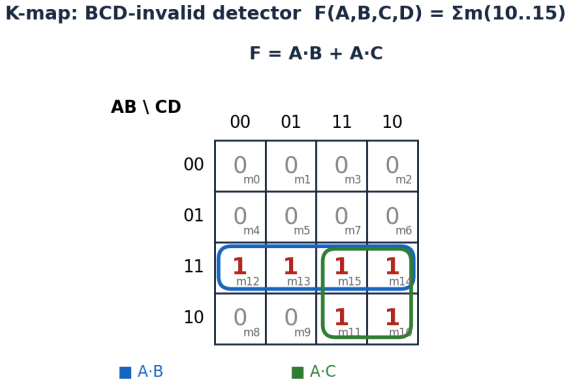
Full adder SUM: checkerboard, no simplification, equals A XOR B XOR Cin.



Full adder COUT = AB + A.Cin + B.Cin.



2:1 MUX: Y = S'A + SB (AB is a redundant hazard-cover term).



BCD-invalid detector: F = AB + AC.

3.5 Truth tables of the arithmetic and selection circuits

A	B	SUM	CARRY
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

A	B	Cin	SUM	COUT
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

SEL	A	B	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Left to right: half adder, full adder, 2:1 multiplexer. 4:1 mux: SEL=00,01,10,11 selects D0,D1,D2,D3.

4. Linux Environment Setup (Part C)

I did the work on Ubuntu running inside Oracle VirtualBox. The tools come from the normal package manager:

```
sudo apt update
sudo apt install -y iverilog gtkwave git build-essential make nano vim
iverilog -V | head -1
gtkwave --version
```

For editing I used VS Code with the Verilog-HDL/SystemVerilog extension, and Git with GitHub for version control.

Linux File System essentials

Path	Meaning
/	root of the file system
/home/<user>	user's personal files
/usr/bin	installed programs
/etc	configuration files
/tmp	temporary files

Commands practised (Exercise 6)

Command	Use
pwd, ls, cd	where am I, list, change directory
mkdir -p	create directories (nested)
cp, mv, rm	copy, move/rename, delete
cat, nano	view / edit files
chmod	set permissions (e.g. chmod +x script.sh, chmod 644 file)
grep, wc	search text, count lines

The full captured terminal session is in **reports/linux_command_practice_report.md**.

Project folder structure

```
vlsi_task1_logic_library/
|-- rtl/       and_gate.v or_gate.v xor_gate.v nand_gate.v nor_gate.v xnor_gate.v not_gate.v
|              mux2to1.v mux4to1.v half_adder.v full_adder.v
|              ripple_carry_adder_4bit.v adder_nbit.v
|-- tb/        tb_logic_gates.v tb_half_adder.v tb_full_adder.v tb_mux2to1.v tb_mux4to1.v
|              tb_ripple_carry_adder_4bit.v tb_adder_nbit.v
|-- sim/       *.vvp  *.vcd           (generated)
|-- docs/      diagrams/  waveforms_reference/  screenshots/  Project_Documentation.pdf
|-- scripts/   setup_env.sh run_all.sh linux_practice.sh verify_model.py ...
|-- reports/   simulation logs, linux report, daily progress log
|-- Makefile
`-- README.md
```

5. Verilog HDL Design (Part D, F, Exercise 7)

Things I focused on while writing the Verilog: a **module** with named ports; the difference between **wire** (a connection, driven by assign or a module output) and **reg** (a variable assigned inside always or initial, which does not always turn into a flip-flop); operators such as `& | ^ ~ ?: {} +`; `assign` for simple combinational logic; `always @(*)` when I need a case statement; `initial` only in testbenches; and `parameter` with `generate` to make the adder any width.

Coding standards followed

• One module per file; file name equals module name.

• snake_case identifiers; consistent port order (inputs, then outputs).

• Every `case` has a `default`; sensitivity list is `@(*)` - no unintended latches.

• No delays or non-synthesizable constructs in RTL.

• Named port connections (`.a(x)`) on every instance.

Module listings

rtl/and_gate.v

```
//==============================================================================
// Module  : and_gate
// Purpose : 2-input AND gate  (y = a & b)
// Style   : Verilog-2001, synthesizable, no delays in RTL
//==============================================================================
`timescale 1ns/1ps

module and_gate (
    input  wire a,
    input  wire b,
    output wire y
);

assign y = a & b;

endmodule
```

rtl/half_adder.v

```
//==============================================================================
// Module  : half_adder
// Purpose : Half adder: sum = a^b, carry = a&b
// Style   : Verilog-2001, synthesizable, no delays in RTL
//==============================================================================
`timescale 1ns/1ps

module half_adder (
    input  wire a,
    input  wire b,
    output wire sum,
    output wire carry
);

assign sum   = a ^ b;
assign carry = a & b;

endmodule
```

rtl/full_adder.v

```
//==============================================================================
// Module  : full_adder
// Purpose : Full adder built structurally from two half adders and an OR gate
// Style   : Verilog-2001, synthesizable, no delays in RTL
//==============================================================================
`timescale 1ns/1ps

module full_adder (
    input  wire a,
    input  wire b,
```

```
    input  wire cin,
    output wire sum,
    output wire cout
);

wire s1, c1, c2;

half_adder ha0 (.a(a),  .b(b),   .sum(s1),  .carry(c1));
half_adder ha1 (.a(s1), .b(cin), .sum(sum), .carry(c2));

assign cout = c1 | c2;

endmodule
```

rtl/mux4to1.v

```
//============================================================================
// Module  : mux4to1
// Purpose : 4:1 multiplexer using a combinational always block
// Style   : Verilog-2001, synthesizable, no delays in RTL
//============================================================================
`timescale 1ns/1ps

module mux4to1 (
    input  wire       d0,
    input  wire       d1,
    input  wire       d2,
    input  wire       d3,
    input  wire [1:0] sel,
    output reg        y
);

// Full sensitivity list (@*) and default branch -> no inferred latch
always @(*) begin
    case (sel)
        2'b00:   y = d0;
        2'b01:   y = d1;
        2'b10:   y = d2;
        2'b11:   y = d3;
        default: y = 1'bx;
    endcase
end

endmodule
```

rtl/ripple_carry_adder_4bit.v

```
//============================================================================
// Module  : ripple_carry_adder_4bit
// Purpose : 4-bit ripple carry adder from four full adders
// Style   : Verilog-2001, synthesizable, no delays in RTL
//============================================================================
`timescale 1ns/1ps

module ripple_carry_adder_4bit (
    input  wire [3:0] a,
    input  wire [3:0] b,
    input  wire       cin,
    output wire [3:0] sum,
    output wire       cout
);

wire c1, c2, c3;   // internal carry chain

full_adder fa0 (.a(a[0]), .b(b[0]), .cin(cin), .sum(sum[0]), .cout(c1));
full_adder fa1 (.a(a[1]), .b(b[1]), .cin(c1),  .sum(sum[1]), .cout(c2));
full_adder fa2 (.a(a[2]), .b(b[2]), .cin(c2),  .sum(sum[2]), .cout(c3));
full_adder fa3 (.a(a[3]), .b(b[3]), .cin(c3),  .sum(sum[3]), .cout(cout));

endmodule
```

rtl/adder_nbit.v

```
//============================================================================
// Module  : adder_nbit
// Purpose : Parameterized N-bit ripple carry adder (generate loop)
// Style   : Verilog-2001, synthesizable, no delays in RTL
//============================================================================
`timescale 1ns/1ps
```

```
module adder_nbit #(
    parameter N = 8
) (
    input  wire [N-1:0] a,
    input  wire [N-1:0] b,
    input  wire         cin,
    output wire [N-1:0] sum,
    output wire         cout
);

wire [N:0] carry;          // carry[0] = cin, carry[N] = cout
assign carry[0] = cin;

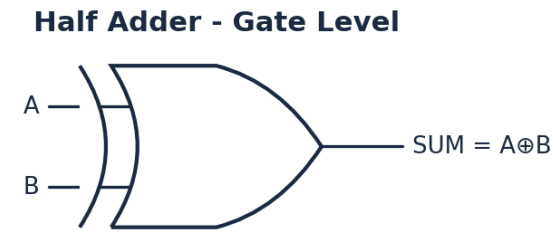
genvar i;
generate
    for (i = 0; i < N; i = i + 1) begin : g_fa
        full_adder fa (
            .a   (a[i]),
            .b   (b[i]),
            .cin (carry[i]),
            .sum (sum[i]),
            .cout(carry[i+1])
        );
    end
endgenerate

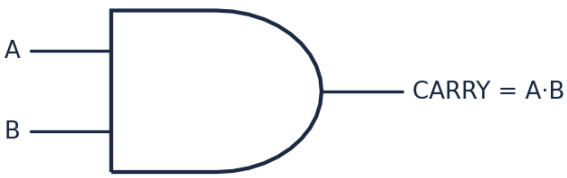
assign cout = carry[N];

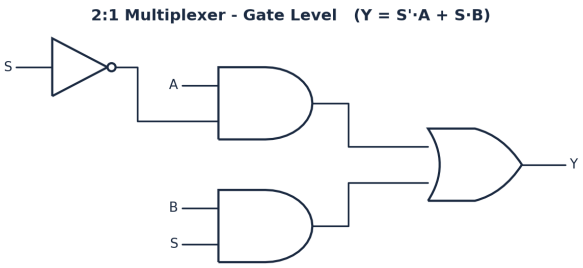
endmodule
```

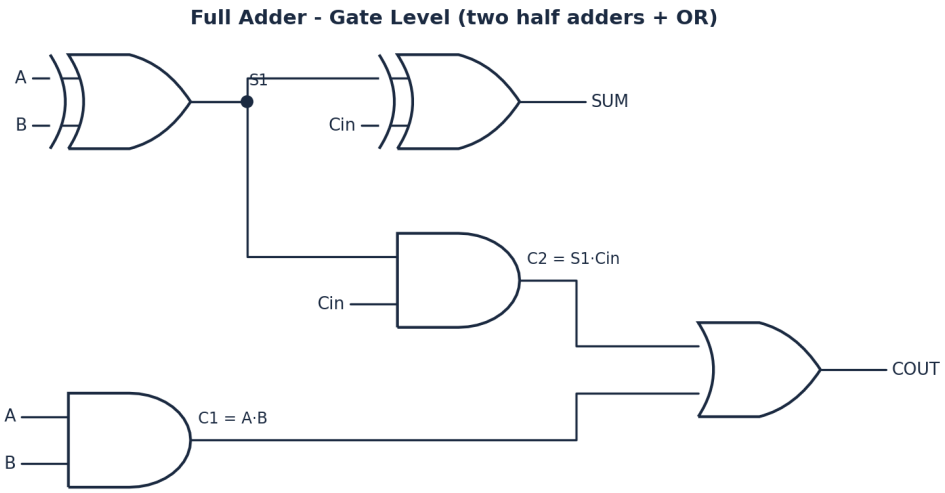
The remaining gate modules (or, xor, nand, nor, xnor, not) and mux2to1 follow the same pattern and are in the **rtl/** folder.

Circuit diagrams

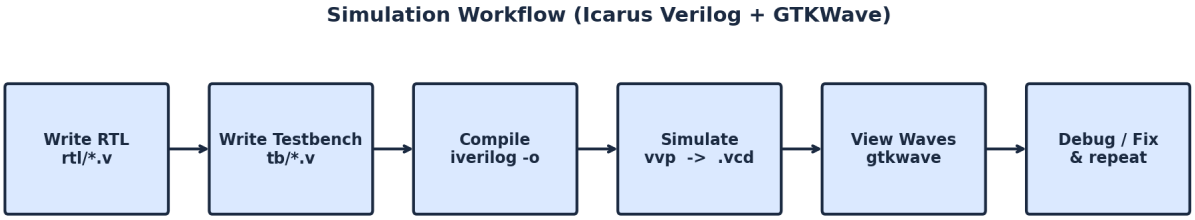







Full Adder - Gate Level (two half adders + OR)
A
B
S1
Cin
SUM
C2 = S1·Cin
Cin
COUT
A
B
C1 = A·B

6. Simulation and Testbench Development (Part E)



A testbench is a module with no ports. It creates the circuit under test, applies inputs from an `initial` block, compares the outputs with the values I expect, and then stops with `$finish`. The `$dumpfile` and `$dumpvars` lines write a VCD file so GTKWave can show the waves. I compare with `!==` instead of `!=` so that an unknown (x) output also counts as a failure.

Commands

```
iverilog -g2005 -Wall -o sim/tb_full_adder.vvp tb/tb_full_adder.v rtl/*.v
vvp sim/tb_full_adder.vvp
gtkwave sim/tb_full_adder.vcd
```

Test strategy

Testbench	DUT	Stimulus	Checks
tb_logic_gates	7 gates	all 4 input pairs	each output vs. operator expression
tb_half_adder	half_adder	4 vectors	sum, carry
tb_full_adder	full_adder	8 vectors	{cout,sum} == a+b+cin
tb_mux2to1	mux2to1	8 vectors	y == (sel ? b : a)
tb_mux4to1	mux4to1	64 vectors (4 sel x 16 data)	y == d[sel]
tb_ripple_carry_adder_4bit	4-bit RCA	5 directed + 512 exhaustive	{cout,sum} == a+b+cin
tb_adder_nbit	adder_nbit N=8, N=16	corner cases + 200 random each	{cout,sum} == a+b+cin

tb/tb_full_adder.v

```
`timescale 1ns/1ps
module tb_full_adder;

reg  a, b, cin;
wire sum, cout;
reg  [1:0] expected;     // {cout, sum}
integer i, errors;

full_adder dut (.a(a), .b(b), .cin(cin), .sum(sum), .cout(cout));

initial begin
    $dumpfile("sim/tb_full_adder.vcd");
    $dumpvars(0, tb_full_adder);
    errors = 0;
    $display(" a b cin | sum cout");
    for (i = 0; i < 8; i = i + 1) begin
        {a, b, cin} = i[2:0];
        #10;
        expected = a + b + cin;            // 2-bit context -> no overflow
        $display(" %b %b  %b  |  %b    %b", a, b, cin, sum, cout);
        if ({cout, sum} !== expected) begin
            errors = errors + 1;
            $display("  FAIL: got {cout,sum}=%b expected=%b", {cout, sum}, expected);
        end
    end
    if (errors == 0) $display("RESULT: tb_full_adder PASSED");
    else             $display("RESULT: tb_full_adder FAILED (%0d errors)", errors);
    #10 $finish;
```

```
end

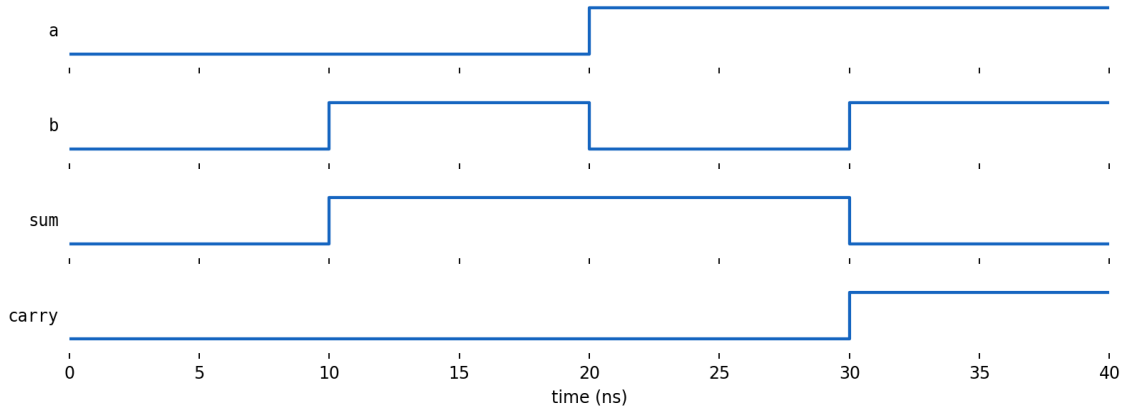
endmodule
```

7. Waveform Verification

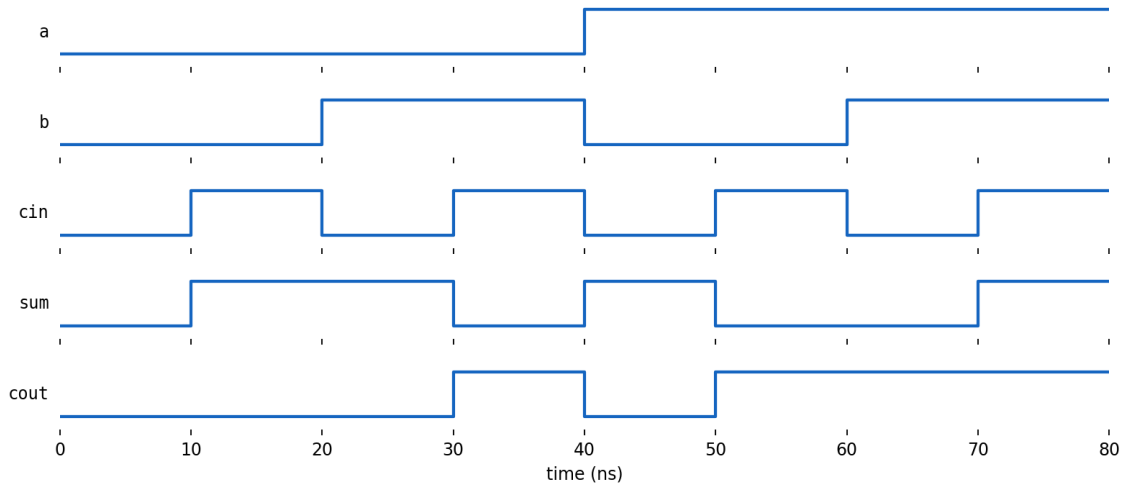
> The plots below are **reference waveforms** that I generated with a Python model to show what a correct run looks like. They are not GTKWave captures. My own GTKWave screenshots follow after them.

Reading the full adder plot: **sum** is high when an odd number of inputs are high, and **cout** is high when at least two inputs are high. For the 4-bit adder, cout is 1 exactly when a + b + cin is greater than 15. For example, 15 + 1 gives 0 with a carry out, and 15 + 15 + 1 = 31 gives 1111 with a carry out.

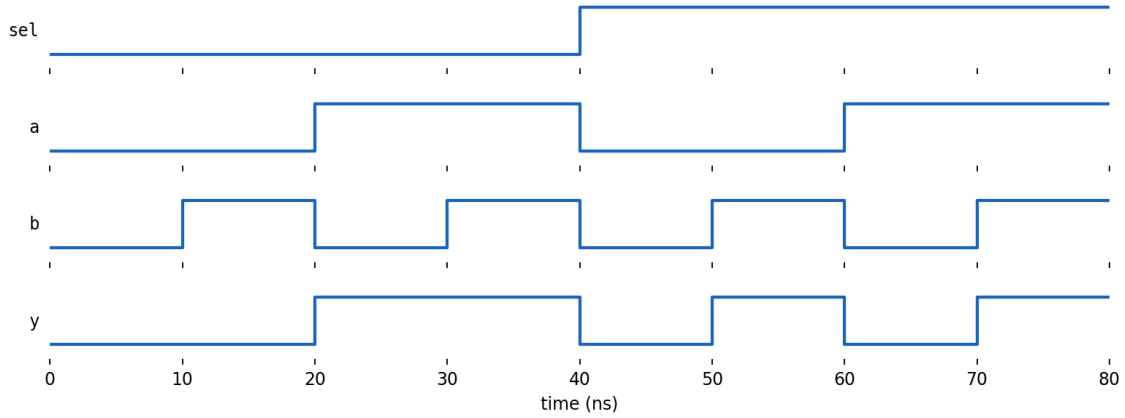
tb_half_adder [reference plot from Python golden model - replace with your GTKWave screenshot]



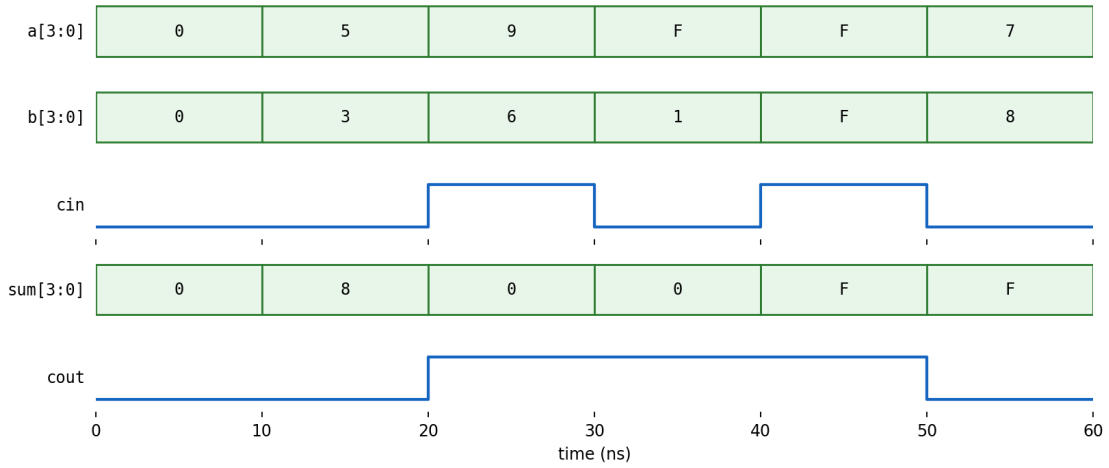
tb_full_adder [reference plot from Python golden model - replace with your GTKWave screenshot]



tb_mux2to1 [reference plot from Python golden model - replace with your GTKWave screenshot]

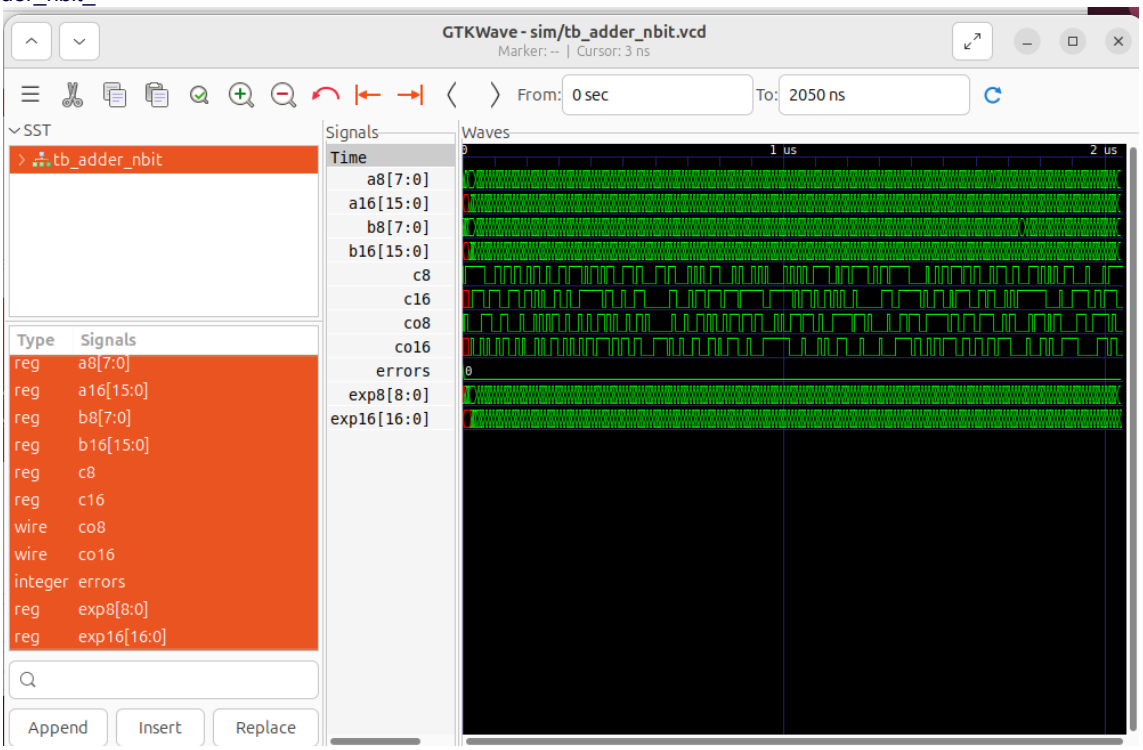


tb_ripple_carry_adder_4bit [reference plot from Python golden model - replace with your GTKWave screenshot]

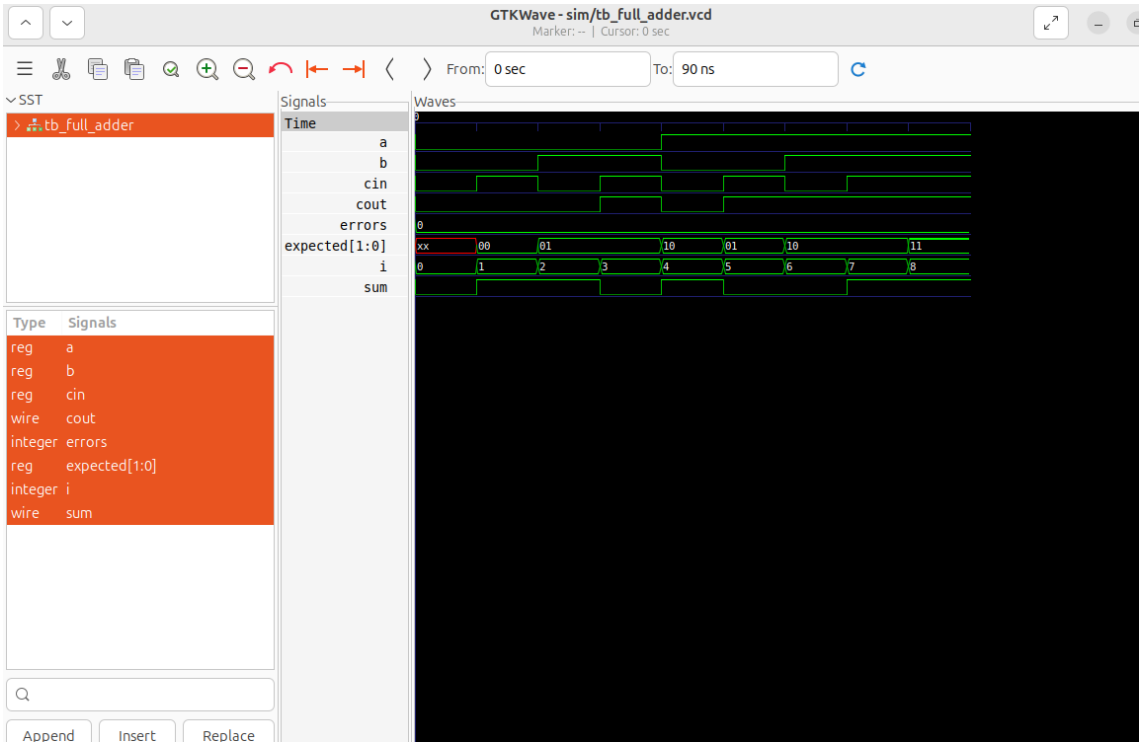


GTKWave screenshots

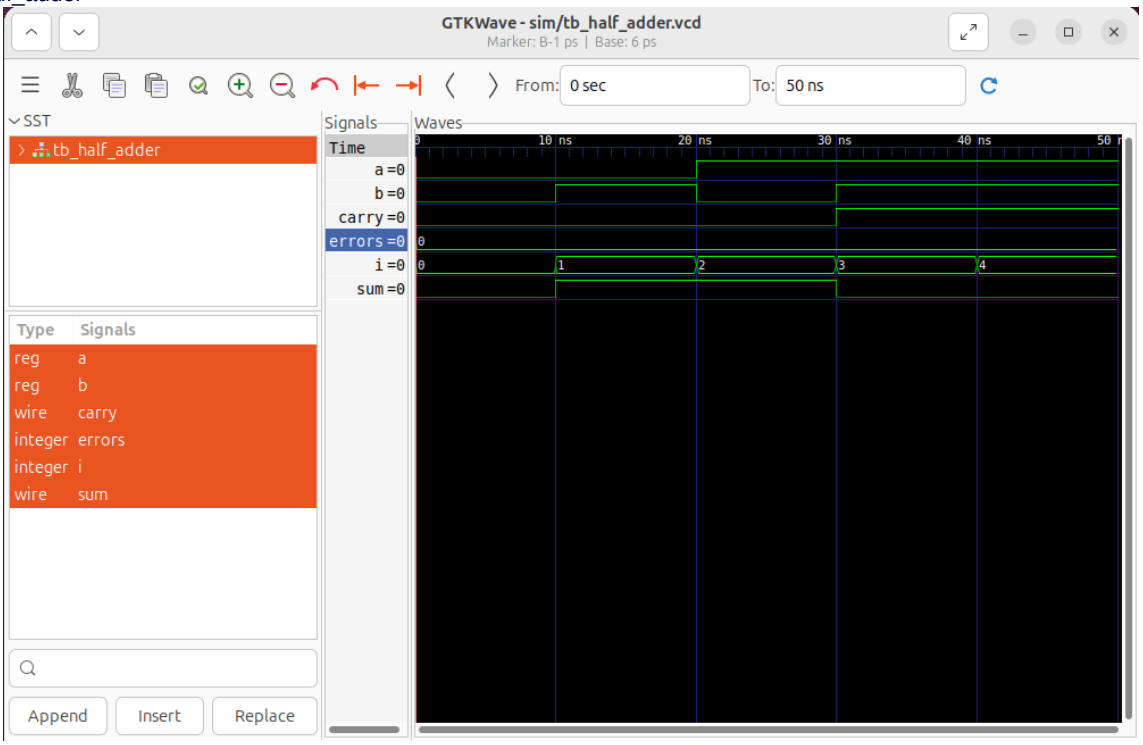
tb_adder_nbit_



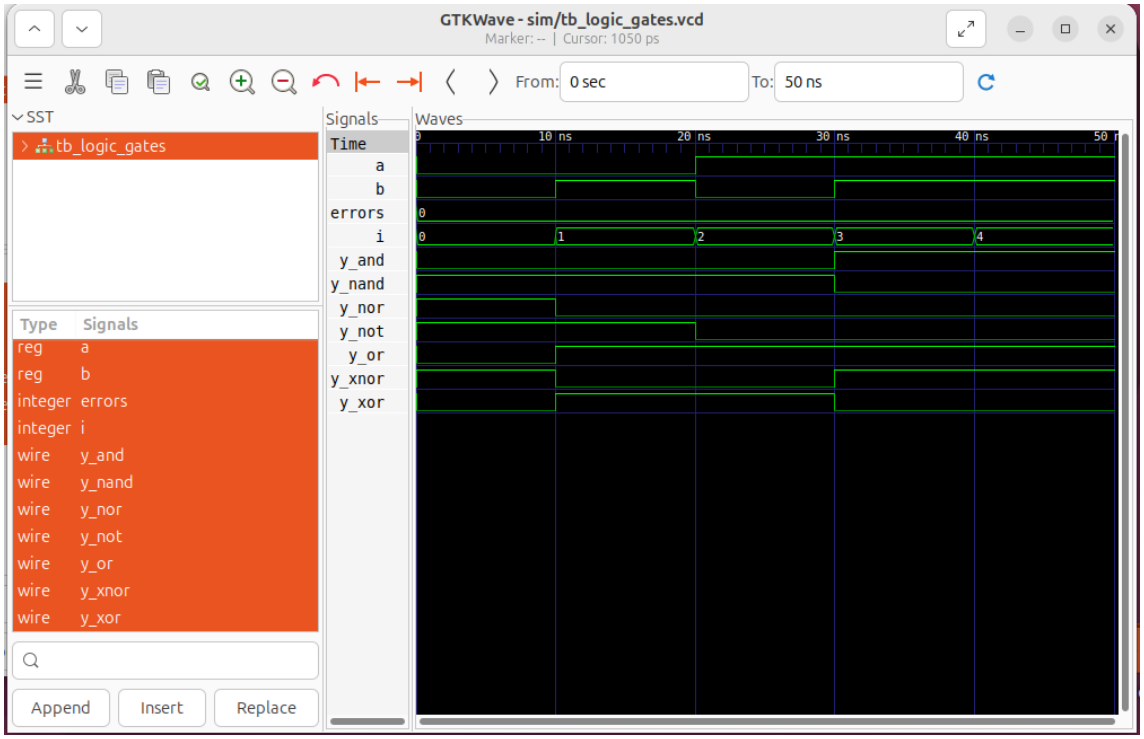
tb_full_adder



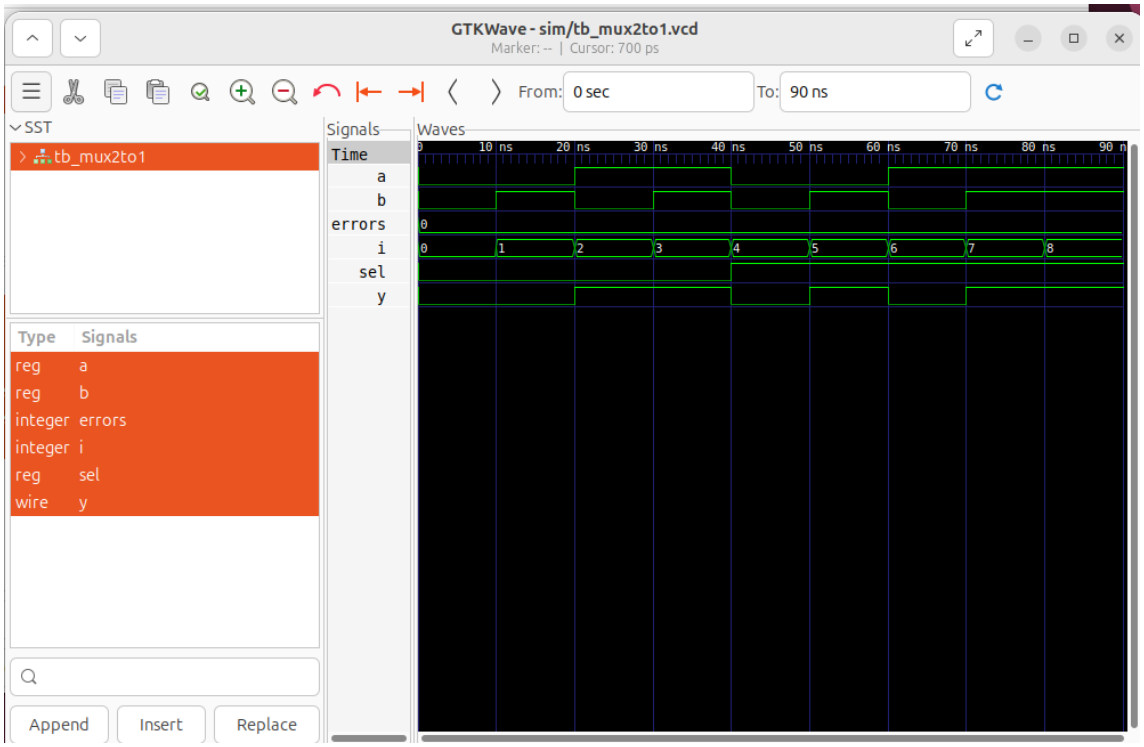
tb_half_adder



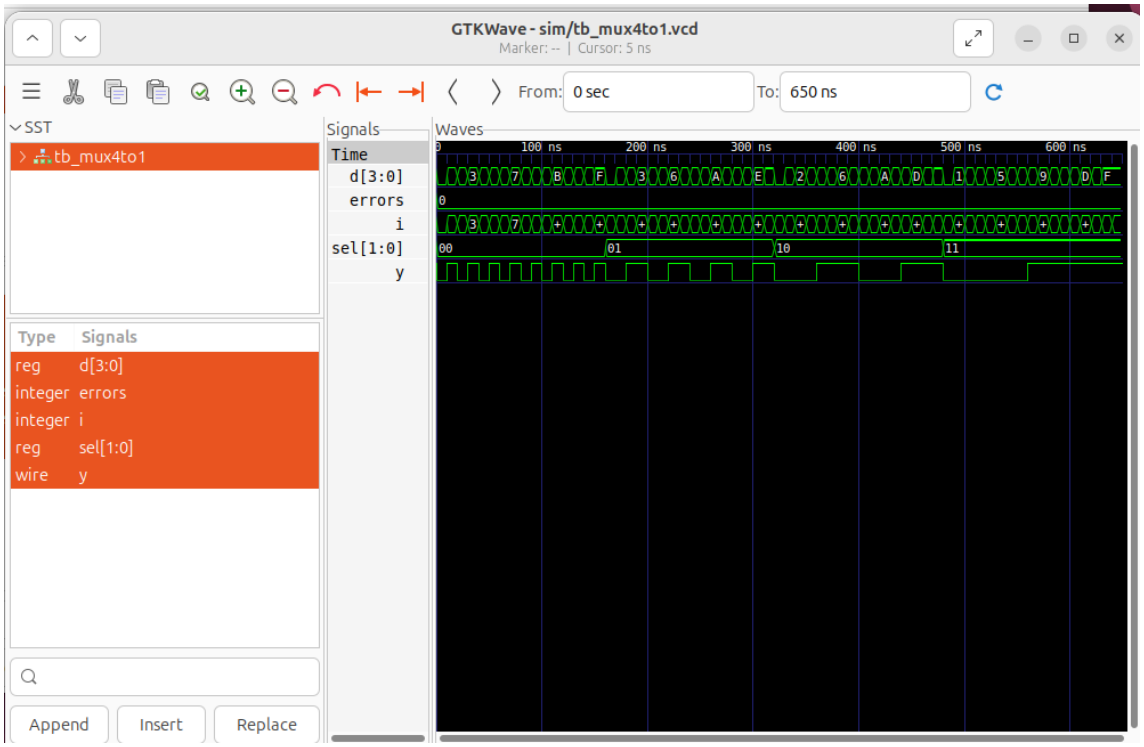
tb_logic_gates



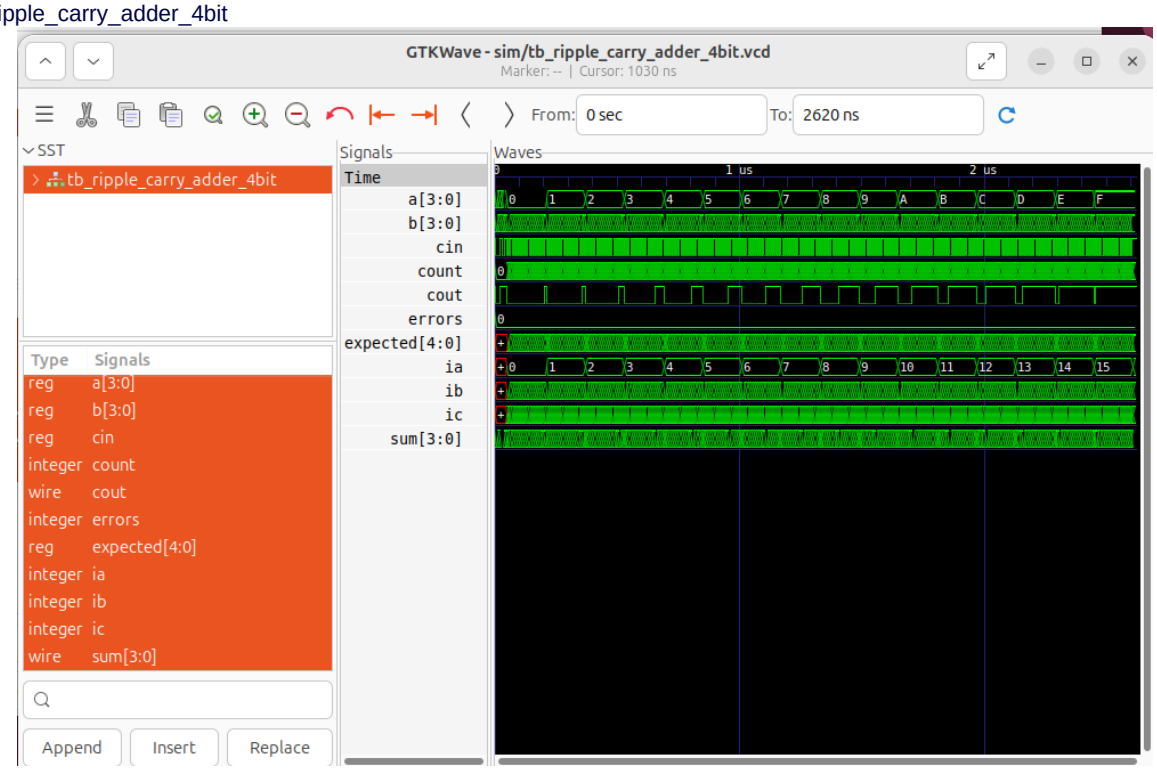
tb_mux2to1



tb_mux4to1



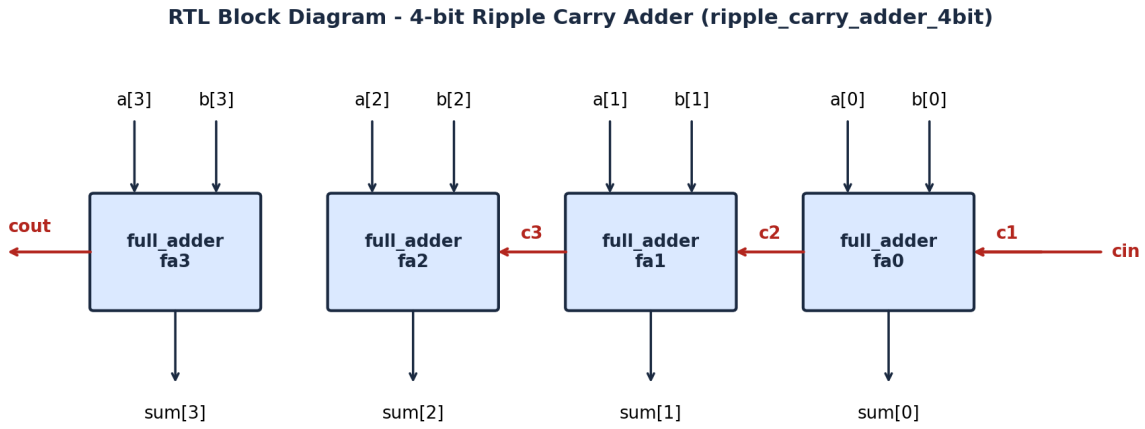
tb_ripple_carry_adder_4bit



Simulation results

Testbench	Expected console result	My result (Icarus Verilog 12.0, Ubuntu)
tb_logic_gates	RESULT: tb_logic_gates PASSED	PASSED
tb_half_adder	RESULT: tb_half_adder PASSED	PASSED
tb_full_adder	RESULT: tb_full_adder PASSED	PASSED
tb_mux2to1	RESULT: tb_mux2to1 PASSED	PASSED
tb_mux4to1	RESULT: tb_mux4to1 PASSED	PASSED
tb_ripple_carry_adder_4bit	RESULT: tb_ripple_carry_adder_4bit PASSED	PASSED
tb_adder_nbit	RESULT: tb_adder_nbit PASSED	PASSED

8. Mini RTL Design: 4-bit Ripple Carry Adder (Part F)



Carry ripples from fa0 (right, LSB) to fa3 (left, MSB). Critical path = 4 x (full-adder carry delay).

The 4-bit adder is four full adders in a row. The carry-out of each stage goes into the carry-in of the next one. Each sum bit is a XOR b XOR carry-in, and the next carry is a.b + carry-in.(a XOR b). The good part is that it is small and simple. The downside is that the carry has to travel through every stage, so it gets slower as the adder gets wider. Faster designs such as carry-lookahead fix this, and I expect to meet them in later tasks.

Parameterized N-bit adder (Exercise 7)

The parameterized adder, `adder_nbit #(.N(16))`, uses a `generate for` loop to create N full adders and a carry wire `wire [N:0] carry`. The same file gave me an 8-bit and a 16-bit adder in tb_adder_nbit.

9. Mini Project Summary (Part H)

Digital Arithmetic Logic Circuit Library Using Verilog HDL brings everything together: gates, half and full adders, multiplexers, the ripple carry adder, testbenches and waveform checks. Everything runs with `make` or `scripts/run_all.sh`, and all 7 testbenches passed in my Icarus Verilog run.

Link to the capstone

The capstone needs an ALU. I can use `adder_nbit` with N = 32 as the adder inside it, the multiplexers to choose between results, and the same style of self-checking testbench for every new block.

10. Conclusion and Future Work

In this task I went from Boolean logic and K-maps to Verilog modules, then to self-checking testbenches and waveforms, and finally to an organised project with a Makefile, README and Git. I made a few typing mistakes in the terminal along the way, and my first GTKWave view was zoomed in too far to show anything, so I learned to read error messages and zoom properly. Next I will move on to sequential circuits: flip-flops, counters, shift registers and FSMs.

References

• Icarus Verilog documentation (iverilog.fandom.com / steveicarus.github.io/iverilog)

• GTKWave documentation (gtkwave.sourceforge.net)

• IEEE Std 1364-2005, Verilog Hardware Description Language

• HDLBits (hdlbits.01xz.net) - practice problems

- Task sheet: Sumerix Global VLSI Internship Task-1